

1. I2C 地址

I2C 设备地址用于在 I2C 总线上唯一标识一个设备。通常，它是 7 位或 10 位长。在通信中，地址和读/写位（1 位）一起构成一个完整的 8 位数据。I2C 地址的格式通常是：

```
| 7-bit Address | R/W |
```

其中 7-bit Address 是设备地址，R/W 表示读/写操作。设备手册会提供具体的地址信息。

2. ADC 电池采样电压处理公式

ADC（模拟到数字转换器）用于将模拟电压信号转换为数字值。假设你有一个 12 位的 ADC，电池的电压信号范围是 0 到 3.3V。采样到的数字值可以通过以下公式转换为实际电压值：

$$V_{battery} = \left(\frac{ADC_value}{ADC_max} \right) \times V_{ref}$$

($V_{battery}$) 是电池电压。

(ADC_value) 是 ADC 读数。

(ADC_max) 是 ADC 的最大值，对于 12 位 ADC，通常是 4095。

(V_{ref}) 是 ADC 的参考电压。

3. 设计一个 FIFO

FIFO（先进先出）队列是一种数据结构，遵循先进先出的原则。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define SIZE 10 // 队列大小

typedef struct {
    int data[SIZE];
    int front, rear, count;
} FIFOQueue;

void initQueue(FIFOQueue *q) {
    q->front = 0;
    q->rear = 0;
    q->count = 0;
}

bool isFull(FIFOQueue *q) {
    return q->count == SIZE;
}

bool isEmpty(FIFOQueue *q) {
    return q->count == 0;
}

void enqueue(FIFOQueue *q, int value) {
    if (isFull(q)) {
```

```

        printf("Queue is full\n");
        return;
    }
    q->data[q->rear] = value;
    q->rear = (q->rear + 1) % SIZE;
    q->count++;
}

int dequeue(FIFOQueue *q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return -1; // Error value
    }
    int value = q->data[q->front];
    q->front = (q->front + 1) % SIZE;
    q->count--;
    return value;
}

int main() {
    FIFOQueue q;
    initQueue(&q);

    enqueue(&q, 1);
    enqueue(&q, 2);

    printf("Dequeued: %d\n", dequeue(&q));
    printf("Dequeued: %d\n", dequeue(&q));

    return 0;
}

```

4. 多线程的注意事项

在多线程项目中，以下几点是关键：

- **线程同步**：使用互斥锁（mutexes）和条件变量（condition variables）来防止数据竞争。
- **死锁避免**：确保锁的获取顺序一致，避免环形等待。
- **资源共享**：避免线程间对共享资源的冲突访问。
- **线程安全**：保证代码在多线程环境下能够正确执行。
- **性能优化**：避免过多线程创建和销毁，使用线程池等技术。

5. 指针

指针是一个变量，用于存储另一个变量的内存地址。在 C 语言中，指针用于动态内存分配、数组和字符串操作、函数参数传递等。以下是一些常见的指针操作示例：

```
int main() {
    int value = 10;
    int *ptr = &value; // 指针指向变量 value 的地址

    printf("Value: %d\n", value);
    printf("Pointer value: %p\n", (void*)ptr);
    printf("Dereferenced pointer value: %d\n", *ptr);

    return 0;
}
```

6. Linux 编程和 Linux 指令

Linux 编程通常涉及系统调用、文件操作、进程管理等。常用的 Linux 指令包括：

- `ls`：列出目录内容
- `cd`：更改目录
- `cp`：复制文件
- `mv`：移动文件
- `rm`：删除文件
- `grep`：搜索文本
- `ps`：查看进程状态
- `top`：实时查看系统资源

7. STM32 启动流程

STM32 启动流程如下：

复位：上电后，处理器复位。

启动引导加载程序（Bootloader）执行内置的引导程序。

初始化硬件：设置时钟、外设等。

跳转到应用程序：启动主应用程序的入口点。

8. STM32 寄存器的作用

- **GPIO 寄存器**：配置和控制 GPIO 引脚。
- **USART 寄存器**：配置和控制串口通信。
- **TIM 寄存器**：配置和控制定时器。
- **ADC 寄存器**：配置和控制模拟到数字转换器。

9. SPI 调试

SPI（串行外设接口）是一种用于设备间通信的协议。如果你使用示波器调试 SPI，可以：

- 配置 SPI 设备的参数（如时钟极性、相位）。
- 通过 SPI 发送和接收数据。
- 使用示波器或逻辑分析仪查看 SPI 信号的波形。

10. 手撕链表翻转

```
#include <stdio.h>
#include <stdlib.h>
```

```

typedef struct Node {
    int data;
    struct Node *next;
} Node;

void printList(Node *head) {
    while (head != NULL) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

Node* reverseList(Node *head) {
    Node *prev = NULL;
    Node *current = head;
    Node *next = NULL;

    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    return prev;
}

int main() {
    Node *head = (Node*)malloc(sizeof(Node));
    head->data = 1;
    head->next = (Node*)malloc(sizeof(Node));
    head->next->data = 2;
    head->next->next = (Node*)malloc(sizeof(Node));
    head->next->next->data = 3;
    head->next->next->next = NULL;

    printf("Original list:\n");
    printList(head);

    head = reverseList(head);

    printf("Reversed list:\n");
    printList(head);

    return 0;
}

```

11. stm32是大端还是小端？手撕一个判断大端还是小端的程序
使用小端字节序。

```

#include <stdio.h>

int isLittleEndian() {
    unsigned int x = 1;
    char *c = (char*)&x;

```

```
        return (*c == 1);
    }

    int main() {
        if (isLittleEndian()) {
            printf("系统是小端字节序\n");
        } else {
            printf("系统是大端字节序\n");
        }
        return 0;
    }
}
```

如果结果是 1，说明系统是小端字节序。